

Phòng thí nghiệm Bitcoin Script (Lab 03)

Môn học: Blockchain (2026), Khoa Công nghệ Tri thức

Bài thực hành: Tìm hiểu ngôn ngữ kịch bản của Bitcoin

Đề bài gốc: [../Lab03.pdf](#)

Dự án Python minh họa các kiến thức nền tảng về Bitcoin Script trên **Bitcoin testnet**:

- **Bài 1 — P2PKH**: sinh khóa/địa chỉ, khóa tiền và chi tiêu đầu ra.
- **Bài 2 — multisig 2-trên-2 (P2SH)**: tạo redeem script, khóa tiền và chi tiêu bằng hai chữ ký.
- **Bài 3 — Báo cáo**: phân tích, đánh giá và rút ra bài học.

Yêu cầu môi trường

- Windows PowerShell.
- Python 3.10 trở lên và `pip`. Nếu lệnh `python` chưa tồn tại, cài Python từ <https://www.python.org/downloads/windows/>, chọn **Add Python to PATH**, mở lại PowerShell rồi kiểm tra bằng `python --version`.
- Kết nối Internet cho bước cài thư viện và các thao tác với testnet.
- Tùy chọn: Bitcoin Core testnet để chạy thêm `testmempoolaccept`.
- Tùy chọn: Pandoc để xuất báo cáo Markdown sang PDF.

`python-bitcoinlib` cần OpenSSL. Trên Windows, lớp tương thích đi kèm sẽ dò các bản `libcrypto-3` hiện đại, bao gồm bản được cài cùng Git for Windows.

Cài đặt môi trường

Chạy từ thư mục `bitcoin-script-lab`:

```
Set-Location "D:\Coding\Blockchain pract\lab3\bitcoin-script-lab"
python --version
python -m venv code\.venv
.\code\.venv\Scripts\python.exe -m pip install --upgrade pip
.\code\.venv\Scripts\python.exe -m pip install -r code\requirements.txt
```

Các phiên bản phụ thuộc đã được ghim trong `code/requirements.txt`: `python-bitcoinlib==0.12.2` và `requests==2.34.2`. Có thể kích hoạt môi trường ảo để rút gọn lệnh:

```
.\code\.venv\Scripts\Activate.ps1
```

Nếu PowerShell chặn script kích hoạt, không cần đổi execution policy; tiếp tục dùng đường dẫn đầy đủ

`.\code\.venv\Scripts\python.exe` như các lệnh bên dưới.

Cấu hình tùy chọn

```
# Đối Esplora API (bắt buộc HTTPS)
$env:BTC_LAB_ESPLORA_URL = "https://blockstream.info/testnet/api"

# Phí mặc định, đơn vị sat/vb
$env:BTC_LAB_FEE_RATE = "2"

# Đối nơi lưu khóa cục bộ; thư mục này tuyệt đối không được nộp
$env:BTC_LAB_SECRETS_DIR = "D:\duong-dan-rieng\btc-lab-secrets"

# Tùy chọn: bật kiểm tra mempool bằng Bitcoin Core testnet
$env:BTC_LAB_RPC_URL = "http://user:password@127.0.0.1:18332"
```

Mặc định khóa testnet nằm trong `.secrets/wallet.json`, đã được Git và trình đóng gói loại trừ. Thông tin xác thực RPC được xử lý cục bộ và không được in ra màn hình.

Khởi chạy Bài 1 — P2PKH

```
Set-Location "D:\Coding\Blockchain pract\lab3\bitcoin-script-lab\code"

# Sinh hoặc hiển thị lại địa chỉ testnet P2PKH
.\venv\Scripts\python.exe task1_p2pkh.py init

# Chỉ dùng khi giảng viên yêu cầu minh họa WIF; không chụp/đưa WIF vào báo cáo
.\venv\Scripts\python.exe task1_p2pkh.py init --show-private-key
```

Nạp testnet BTC từ faucet vào địa chỉ vừa hiển thị và chờ xác nhận. Sau đó tạo, ký và kiểm tra giao dịch cục bộ (chưa phát lên mạng):

```
.\venv\Scripts\python.exe task1_p2pkh.py spend <DIA_CHI_TESTNET_DICH> 10000 --fee-rate 2
```

Đọc kỹ `raw_transaction`, phí, tiền thừa và trạng thái `local_script_verification`. Chỉ phát giao dịch sau khi kết quả hợp lệ:

```
.\venv\Scripts\python.exe task1_p2pkh.py spend <DIA_CHI_TESTNET_DICH> 10000 --fee-rate 2 --broadcast --wait
```

Nếu UTXO nạp tiền chưa có xác nhận nhưng cần thử nghiệm, thêm `--include-unconfirmed`. `--wait` chỉ hợp lệ khi đi cùng `--broadcast`.

Khởi chạy Bài 2 — P2SH multisig 2-trên-2

```
Set-Location "D:\Coding\Blockchain pract\lab3\bitcoin-script-lab\code"

# Sinh hai cặp khóa, redeem script và địa chỉ P2SH testnet
.\venv\Scripts\python.exe task2_multisig.py init

# Chỉ dùng khi bắt buộc minh họa WIF; không để khóa xuất hiện trong ảnh/báo cáo
.\venv\Scripts\python.exe task2_multisig.py init --show-private-keys
```

Nạp testnet BTC vào địa chỉ P2SH, chờ xác nhận, rồi dry-run trước khi broadcast:

```
.\.venv\Scripts\python.exe task2_multisig.py spend <DIA_CHI_TESTNET_DICH> 10000 --fee-rate 2
.\.venv\Scripts\python.exe task2_multisig.py spend <DIA_CHI_TESTNET_DICH> 10000 --fee-rate 2 --broadcast --wait
```

Unlocking script có dạng `OP_0 <sig1> <sig2> <redeemScript> . OP_0` xử lý lỗi lịch sử của `OP_CHECKMULTISIG` làm lấy thừa một phần tử trên stack.

Cảnh báo: `init --replace` làm mất tham chiếu cục bộ tới khóa cũ. Không chạy tùy chọn này nếu địa chỉ cũ còn tiền. Chỉ sử dụng testnet BTC và địa chỉ đích do nhóm kiểm soát.

Kế hoạch kiểm thử phục vụ báo cáo

Quy trình tuân theo vòng đời an toàn: **lập kế hoạch** → **mô phỏng/kiểm tra cục bộ** → **xác nhận kết quả** → **broadcast có chủ đích** → **chờ xác nhận**. Không broadcast nếu kiểm tra script hoặc `testmempoolaccept` thất bại.

1. Kiểm thử tự động ngoại tuyến

Không cần faucet hay broadcast:

```
Set-Location "D:\Coding\Blockchain pract\lab3\bitcoin-script-lab"

# Toàn bộ test mã giao dịch, ví và lớp mạng mock
.\code\.venv\Scripts\python.exe -m unittest discover -s code\tests -v

# Test riêng theo lớp để ghi kết quả chi tiết vào báo cáo
Push-Location .\code
.\.venv\Scripts\python.exe -m unittest tests.test_wallet -v
.\.venv\Scripts\python.exe -m unittest tests.test_transactions -v
.\.venv\Scripts\python.exe -m unittest tests.test_network -v
Pop-Location

# Test cấu trúc gói nộp bài và việc loại trừ bí mật
.\code\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Các tiêu chí cần ghi nhận:

Nhóm test	Kết quả mong đợi	Minh chứng nên lưu
Ví/địa chỉ	Sinh P2PKH testnet, P2SH 2-trên-2 và lưu/đọc ví thành công	Ảnh terminal đã che dữ liệu nhạy cảm
P2PKH hợp lệ	Chữ ký đúng vượt qua <code>VerifyScript</code>	Tên test và trạng thái <code>ok</code>
P2PKH sai khóa	Chữ ký sai bị từ chối	Negative test trả kết quả mong đợi
Multisig đủ hai chữ ký	Script có dummy <code>OP_0</code> và xác minh thành công	Tên test, mô tả stack
Multisig thiếu chữ ký	Giao dịch 1-trên-2 bị từ chối	Negative test, tuyệt đối không broadcast
Xây dựng giao dịch	Tính đúng phí/tiền thừa, từ chối đầu ra dust	Kết quả test và số liệu phí
Lớp mạng	Phân tích UTXO đúng, chỉ chấp nhận HTTPS, báo rõ khi thiếu UTXO	Kết quả mock test
Đóng gói	Đúng cây <code><GroupID>/Code</code> + <code>Report.pdf</code> ; không có ví/venv	Danh sách file trong ZIP

2. Smoke test CLI ngoại tuyến

```
Set-Location "D:\Coding\Blockchain pract\lab3\bitcoin-script-lab\code"
.\.venv\Scripts\python.exe task1_p2pkh.py --help
.\.venv\Scripts\python.exe task2_multisig.py --help
.\.venv\Scripts\python.exe task1_p2pkh.py init
.\.venv\Scripts\python.exe task2_multisig.py init
```

Xác nhận chương trình chỉ tạo địa chỉ testnet, khóa được lưu trong thư mục bí mật, WIF bị che theo mặc định và hai CLI in đủ hướng dẫn/tham số.

3. Kiểm thử tích hợp testnet cho P2PKH

- Nạp testnet BTC vào địa chỉ của Bài 1 và lưu funding txid/explorer URL.
- Chạy lệnh `spend` không có `--broadcast` ; kiểm tra `local_script_verification: passed` , địa chỉ đích, số satoshi, phí, vsize và change.
- Nếu có Bitcoin Core, xác nhận `testmempoolaccept.allowed = true` .
- Chạy lại với `--broadcast --wait` ; lưu spend txid, thời gian xác nhận và explorer URL.
- Đối chiếu `phí = tổng input - tổng output` , không có output dust.

4. Kiểm thử tích hợp testnet cho multisig

1. Nạp testnet BTC vào địa chỉ P2SH và lưu funding txid/explorer URL.
2. Dry-run giao dịch chi tiêu, xác nhận đủ hai chữ ký và scriptSig bắt đầu bằng `OP_0`.
3. Ghi nhận negative test thiếu một chữ ký từ unit test; không gửi giao dịch lỗi lên mạng.
4. Broadcast giao dịch hợp lệ, chờ xác nhận và lưu spend txid, phí, vsize, explorer URL.

5. Kiểm thử lỗi và an toàn

- Địa chỉ mainnet hoặc địa chỉ đích sai phải bị từ chối.
- Số tiền không đủ, UTXO không đủ hoặc đầu ra dust phải báo lỗi rõ ràng.
- `--wait` không có `--broadcast` phải thất bại.
- Esplora dùng HTTP thường phải bị từ chối; thông tin RPC không xuất hiện trong log.
- Không có WIF, `.secrets`, `wallet.json`, raw credential hoặc ảnh chứa khóa trong Git, báo cáo và ZIP.
- Chỉ đánh dấu hoàn tất sau khi explorer cho thấy các giao dịch testnet đã xác nhận.

6. Bảng minh chứng điền vào báo cáo

Mã	Kịch bản	Kết quả mong đợi	Minh chứng thực tế	Trạng thái
UT-01	Chạy toàn bộ unit test	Tất cả test <code>OK</code>	[ảnh/log]	[Pass/Fail]
E2E-01	Nạp và chi P2PKH	Hai tx testnet được xác nhận	[funding txid, spend txid, URL]	[Pass/Fail]
E2E-02	Nạp và chi P2SH 2-trên-2	Hai tx testnet được xác nhận	[funding txid, spend txid, URL]	[Pass/Fail]
NEG-01	P2PKH sai khóa	<code>VerifyScript</code> từ chối	[log đã làm sạch]	[Pass/Fail]
NEG-02	Multisig thiếu chữ ký	<code>VerifyScript</code> từ chối	[log đã làm sạch]	[Pass/Fail]
PKG-01	Tạo gói nộp	Cấu trúc đúng, không lộ khóa	[danh sách ZIP]	[Pass/Fail]

Xuất báo cáo và đóng gói bài nộp

Điền đầy đủ thông tin nhóm và minh chứng thật vào `report/report.md`; báo cáo cuối không chứa mã nguồn, dùng cỡ chữ 12 pt, tối đa 15 trang. Nếu máy đã cấu hình Pandoc cùng PDF engine, có thể thử:

```
Set-Location "D:\Coding\Blockchain pract\lab3\bitcoin-script-lab"
pandoc report\report.md -o report\Report.pdf -V fontsize=12pt -V geometry:margin=1in
```

Nếu thiếu PDF engine, mở Markdown bằng công cụ soạn thảo phù hợp và xuất thủ công thành đúng tên `report/Report.pdf` . Sau đó tạo gói nộp:

```
.\code\.venv\Scripts\python.exe package_submission.py <GROUP_ID>

# Kiểm tra cây file trong ZIP
Add-Type -AssemblyName System.IO.Compression.FileSystem
[System.IO.Compression.ZipFile]::OpenRead(".\<GROUP_ID>.zip").Entries | Select-Object FullName, Length
```

ZIP hợp lệ chỉ chứa `<GROUP_ID>/Code/...` và `<GROUP_ID>/Report.pdf` .

Tài liệu trong dự án

Tài liệu	Mục đích
docs/01-requirements-analysis.md	Phân tích đầy đủ <code>Lab03.pdf</code> : yêu cầu chức năng/phi chức năng, tiêu chí đánh giá, điểm thiếu trong mã mẫu, giả định và câu hỏi mở
docs/02-objectives-and-scope.md	Mục tiêu, phạm vi, tiêu chí thành công và rủi ro
docs/03-implementation-plan.md	Kiến trúc, kế hoạch 6 giai đoạn, lịch trình, chiến lược kiểm thử và Definition of Done
code/README.md	Hướng dẫn ngắn cho hai quy trình P2PKH và P2SH 2-trên-2
report/report.md	Mẫu báo cáo đúng cấu trúc bài nộp, không chứa mã nguồn

Cấu trúc dự án

```
bitcoin-script-lab/
├── code/           # Mã Python, hai CLI và unit test ngoại tuyến
├── docs/           # Phân tích yêu cầu, phạm vi và kế hoạch triển khai
├── report/report.md # Nguồn báo cáo; xuất thành report/Report.pdf
├── tests/          # Test cho bước đóng gói bài nộp
└── package_submission.py # Tạo <GroupID>.zip theo mục 7 của đề bài
```

Đối chiếu yêu cầu nộp bài (mục 7 của Lab03.pdf)

Yêu cầu	Vị trí đáp ứng
Thư mục <code><Group's ID></code>	Đầu ra của <code>package_submission.py</code>
Thư mục <code><Code></code> chứa toàn bộ dự án	<code>code/</code>
File thực thi (không bắt buộc)	Hai CLI <code>task1_p2pkh.py</code> , <code>task2_multisig.py</code>
<code>Report.pdf</code> tối đa 15 trang, in một mặt, cỡ chữ 12 pt	<code>report/Report.pdf</code>
Thành viên, cấu trúc, ghi chú, tài liệu tham khảo, mô tả vận hành, khó khăn, bài học; không có mã nguồn	Dàn ý trong <code>report/report.md</code>

Trạng thái hiện tại

- ☒ [x] Đã phân tích yêu cầu, mục tiêu và phạm vi.
- ☒ [x] Đã viết kế hoạch triển khai.
- ☒ [x] Đã hoàn thành nền tảng môi trường, lớp giao dịch chung và kiểm tra ngoại tuyến.
- ☒ [x] Đã hoàn thành mã Bài 1; còn thiếu minh chứng faucet/broadcast thực tế.
- ☒ [x] Đã hoàn thành mã Bài 2; còn thiếu minh chứng faucet/broadcast thực tế.
- ☒ [x] Đã tạo mẫu báo cáo; còn thiếu dữ liệu nhóm và kết quả testnet thật.
- ☒ [x] Đã hoàn thành script/test đóng gói; còn thiếu PDF cuối và Group ID.

Lưu ý an toàn

- Tất cả giao dịch chỉ chạy trên **Bitcoin testnet**.
- Khóa riêng testnet nằm trong `.secrets/wallet.json` ; thư mục này bị Git bỏ qua và không được xuất hiện trong ZIP hay ảnh báo cáo.
- Mọi giao dịch được kiểm tra cục bộ bằng Bitcoin Script; khi có Bitcoin Core RPC, chương trình còn gọi `testmempoolaccept` trước khi broadcast.
- Không chia sẻ WIF hoặc seed phrase, kể cả khóa testnet.